



SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Department of Computer Science and Engineering

CSA1304 – Theory of Computation

COURSE ASSIGNMENT REPORT

A Non-deterministic Finite Automaton with Epsilon-Transitions for the Airport Shuttle Control Sequence

Submitted by:

D.Gunavardhan (192372041)

P.Chandu (192372119)

A .Dileep Kumar(192372134)

Under the Supervision

K.Baskar

Course Code & Name: CSA1304 – Theory of Computation

Course Outcomes Mapped: CO2, CO3

SDG Mapping: SDG 9 – Industry, Innovation and Infrastructure

Date of Submission: 02/09/2026

1. Problem Statement and Problem Formulation

An automated airport shuttle operates between four operational states:

- Waiting Area,
- Moving to Terminal,
- Passenger Boarding,
- Returning.

The shuttle receives two types of signals from its environment: p, indicating that a passenger request has been received, and n, indicating that no passenger request has been received.

Initially, the shuttle is stationed in the Waiting Area. When a passenger request (p) is received, the shuttle moves to the Moving to Terminal state and subsequently reaches the Passenger Boarding state. While passengers are boarding, if another passenger request is received, the shuttle may either continue on to serve the new request or remain in the Passenger Boarding state - this is precisely the point at which the system exhibits non-determinism, since two valid continuations exist for the same input symbol from the same state. If no further passenger request is received (n), the shuttle proceeds to the Returning state and finally returns to the Waiting Area, completing one operational cycle.

The formulation task is to design a Nondeterministic Finite Automaton (NFA) whose accepted language is exactly the set of input strings over the alphabet {p,n} that correspond to valid, well-formed sequences of shuttle operations - i.e., sequences that always return the shuttle to a consistent, well-defined state (the Waiting Area) at the point the sequence ends.

2. Objective and Expected Outcomes

2.1 Objective

To formally design, construct, and validate an NFA that models the four-state operational behaviour of the airport shuttle system, correctly capturing the non-deterministic branching that occurs in the Passenger Boarding state, and to demonstrate the automaton's correctness through simulation and structured test cases.

2.2 Expected Outcomes

- A formally specified NFA (5-tuple definition) that models the shuttle's behaviour.
- A state-transition diagram that visually communicates the automaton's structure and its non-deterministic branch.

- A working software simulation of the NFA that can accept or reject arbitrary input sequences.
- A documented set of test cases covering accepted, rejected, and edge-case inputs.
- A reflective account connecting the exercise to the mapped course outcomes and to SDG 9.

3. Requirements, Constraints and Assumptions

3.1 Requirements

- The automaton must have exactly four states corresponding to Waiting Area, Moving to Terminal, Passenger Boarding, and Returning.
- The input alphabet must be restricted to $\{p, n\}$.
- The automaton must exhibit genuine non-determinism at the Passenger Boarding state on input p .
- The automaton must accept exactly those strings that represent a shuttle returning to a well-defined resting condition.

3.2 Constraints

- The shuttle controller has two input signals: p and n .
- ϵ (epsilon) is not an input symbol; it represents an automatic transition without consuming input.
- Therefore, the input alphabet is $\Sigma = \{p, n\}$, while epsilon transitions are used only for automatic state changes.

3.3 Assumptions

- The transition from Moving to Terminal to Passenger Boarding, and from Returning back to Waiting Area, happens automatically once the shuttle physically completes that leg of the journey, and is not gated on a fresh p/n signal. This is modelled using epsilon (silent) transitions.
- The shuttle begins every session in the Waiting Area.
- A sequence of operations is considered 'valid' if, after all input symbols are consumed, the shuttle can be in the Waiting Area along at least one of its non-deterministic computation paths.

4. Application of Relevant Course Knowledge / Concepts

This assignment directly applies concepts from the Theory of Computation syllabus mapped to CO2 (regular expressions and formal languages) and CO3 (design of finite automata). The core concepts used are:

- Formal definition of an NFA as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$.
- The transition function $\delta: Q \times \Sigma \rightarrow 2^Q$, whose range is a set of states rather than a single state, which is what allows non-determinism to be expressed.
- Epsilon (ϵ) transitions, which let an NFA change state without consuming an input symbol - used here to model the automaton's automatic state changes.
- Epsilon-closure, the standard technique used to simulate an ϵ -NFA by expanding a set of states to include everything reachable via ϵ -moves before and after each input symbol.
- Language acceptance for an NFA: a string w is accepted if there exists at least one computation path from q_0 to some state in F on w .

5. Design / Proposed Solution / Methodology

The shuttle's four operational states map directly onto four NFA states. Each state and its associated signals were analysed individually to derive the transition function, with special attention paid to the

5.1 Formal Definition

The NFA is defined as $M = (Q, \Sigma, \delta, q_0, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$, corresponding to Waiting Area, Moving to Terminal, Passenger Boarding, and Returning respectively.
- $\Sigma = \{ p, n \}$
- q_0 = Waiting Area (start state)
- $F = \{ q_0 \}$ (a sequence is valid once the shuttle is back in the Waiting Area)
- δ is given in the transition table below (Table 5.1).

Table 5.1 - NFA transition table

Current State	Input	Next State(s)	Nature
q0 (Waiting Area)	p	{ q1 }	Deterministic
q0 (Waiting Area)	n	{ q0 }	Deterministic
q1 (Moving to Terminal)	ϵ	{ q2 }	Automatic (epsilon move)
q2 (Passenger Boarding)	p	{ q1, q2 }	Non-deterministic (branch point)
q2 (Passenger Boarding)	n	{ q3 }	Deterministic
q3 (Returning)	ϵ	{ q0 }	Automatic (epsilon move)

5.2 State-Transition Diagram

Figure 5.1 shows the resulting state diagram. The double circle marks the accepting start state (q0). Solid black arrows are ordinary input-driven transitions; the dashed grey arrows marked ϵ are the automatic (epsilon) transitions; the red arrows mark the non-deterministic branch at q2, where a single input symbol P leads to two different next states.

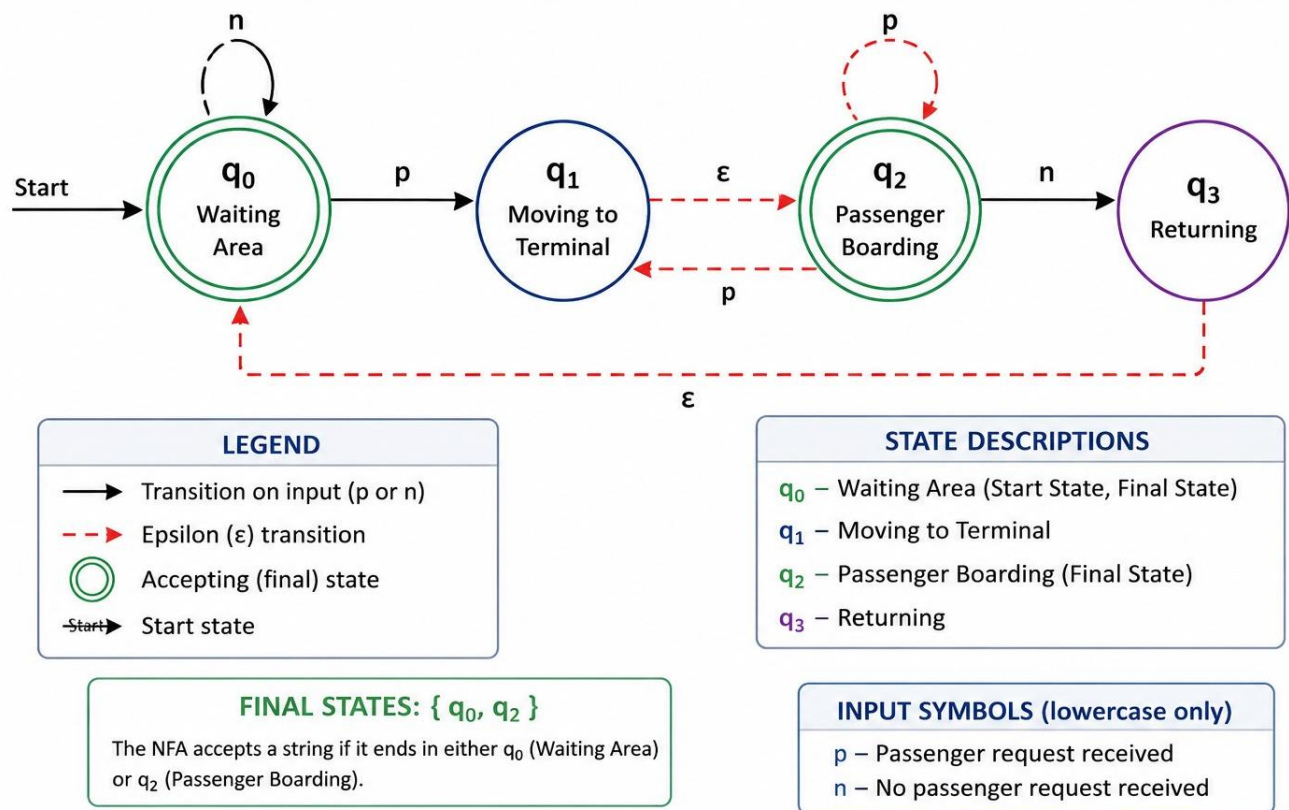


Figure 5.1 - State-transition diagram of the airport shuttle NFA

5.3 Why This Is Genuinely Non-Deterministic

At state q_2 (Passenger Boarding), on input p , the transition function returns the set $\{q_1, q_2\}$ rather than a single state. This means that on receiving a new passenger request while boarding, the automaton has two equally valid continuations available for the very same input symbol from the very same state - it may transition to q_1 (go and serve the new request) or remain at q_2 (continue the current boarding). No amount of additional lookahead within a DFA's single-state model can represent this without first converting the automaton (e.g., via the subset construction), which is exactly why the

6. Algorithm / Pseudocode

The simulation algorithm below implements standard ε -NFA execution using epsilon-closures, so that the automatic (silent) transitions are correctly folded into every step.

```
ALGORITHM SimulateNFA(inputString)
currentStates  $\leftarrow$  EpsilonClosure( $\{q_0\}$ )
FOR EACH symbol IN inputString:
    IF symbol NOT IN  $\{p, n\}$ :
        RETURN REJECT // Invalid symbol
    rawNext  $\leftarrow$   $\{\}$ 
    FOR EACH state IN currentStates:
        rawNext  $\leftarrow$  rawNext UNION  $\delta(\text{state}, \text{symbol})$ 
    currentStates  $\leftarrow$  EpsilonClosure(rawNext)
    IF currentStates =  $\{\}$ :
        BREAK // Dead configuration
IF currentStates INTERSECT F  $\neq$   $\{\}$ :
    RETURN ACCEPT
ELSE:
    RETURN REJECT
FUNCTION EpsilonClosure(states)
closure  $\leftarrow$  states
stack  $\leftarrow$  states
WHILE stack is not empty:
    s  $\leftarrow$  stack.pop()
    FOR EACH t IN  $\delta(s, \varepsilon)$ :
        IF t NOT IN closure:
            closure  $\leftarrow$  closure UNION  $\{t\}$ 
            stack.push(t)
RETURN closure
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Tool / Environment	Purpose
Python 3	Programming language used
Python IDLE	Writing, editing, and executing the Python program
GitHub	Storing and managing the source code

7.2 Source Code

The complete simulator, including the formal transition table encoded as a Python dictionary and a driver that runs the test suite in Section 8, is listed below.

```
# AIRPORT SHUTTLE NFA SIMULATOR
```

```
# ACCEPTS ONLY LOWERCASE INPUTS: p and n
```

```
STATES = {"q0", "q1", "q2", "q3"}
```

```
ALPHABET = {"p", "n"}
```

```
START_STATE = "q0"
```

```
FINAL_STATES = {"q0"}
```

```
EPS = "eps"
```

```
TRANSITIONS = {
```

```
    ("q0", "p"): {"q1"},
```

```
    ("q0", "n"): {"q0"},
```

```
    # Automatic transition
```

```
    ("q1", EPS): {"q2"},
```

```
    # Non-deterministic transition
```

```
    ("q2", "p"): {"q1", "q2"},
```

```
    ("q2", "n"): {"q3"},
```

```
    # Automatic return
```

```
    ("q3", EPS): {"q0"},
```

```
}
```



```

# EPSILON CLOSURE
def epsilon_closure(states):
    stack = list(states)
    closure = set(states)
    while stack:
        state = stack.pop()
        for next_state in TRANSITIONS.get((state, EPS), set()):
            if next_state not in closure:
                closure.add(next_state)
                stack.append(next_state)
    return closure

# NFA STEP
def nfa_step(current_states, symbol):
    next_states = set()
    for state in current_states:
        next_states |= TRANSITIONS.get(
            (state, symbol),
            set()
        )
    return epsilon_closure(next_states)

# SIMULATE NFA
def simulate(input_string):
    current_states = epsilon_closure({START_STATE})
    print("\nInput String:", input_string)
    print(
        "Step 0:",
        current_states
    )
    for i, symbol in enumerate(input_string, start=1):
        # Check whether symbol belongs to alphabet
        if symbol not in ALPHABET:
            print(
                f"\nREJECTED: '{symbol}' is invalid."
            )
            print(
                "Only lowercase 'p' and 'n' are allowed."
            )
            return False
    current_states = nfa_step(
        current_states,
        symbol
    )

```

```

    )
    print(
        f"Step {i}: On '{symbol}' ->",
        current_states
    )

# Check acceptance
if current_states & FINAL_STATES:
    print("\nRESULT: ACCEPTED")
    return True
else:
    print("\nRESULT: REJECTED")
    return False

# MAIN PROGRAM
if __name__ == "__main__":

    print("=" * 50)
    print("AIRPORT SHUTTLE NFA SIMULATOR")
    print("=" * 50)
    print("\nAllowed input symbols:")
    print("p -> Passenger request")
    print("n -> No passenger request")
    print("\nOnly lowercase letters are allowed!")
    while True:
        user_input = input(
            "\nEnter input string (or 'exit' to quit): "
        ).strip()
        # Exit
        if user_input == "exit":
            print("\nProgram Closed.")
            break
        # Check empty input
        if user_input == "":
            print(
                "\nEmpty input entered."
            )
            simulate("")

        else:
            simulate(user_input)

    # Continue
    choice = input(

```

```

        "\nDo you want to try again? (y/n): "
    ).strip()

    if choice != "y":
        print(
            "\nThank you!"
        )
        Break

```

8. Test Cases and Expected/Actual Results

Twelve representative input strings were chosen to exercise every transition in the automaton, including the empty string, purely idle sequences, single and multiple non-deterministic branches at q2, complete and incomplete cycles, back-to-back cycles, and an invalid-symbol case. Table 8.1 lists each case together with the automaton's actual result (produced by the simulator in Section 7.2) and a short interpretation.

Table 8.1 - Test case documentation

Input String	Result	Interpretation
ϵ (empty)	Accepted	Shuttle idles in the Waiting Area and never leaves.
n	Accepted	No request arrives while waiting; shuttle stays in the Waiting Area.
nnnn	Accepted	Several idle cycles occur with no passenger request.
p	Rejected	Shuttle leaves for the terminal, but the cycle is not completed.
pn	Accepted	One passenger request is handled and the shuttle completes one full cycle.
ppn	Accepted	An additional passenger request occurs during boarding before the cycle closes.
ppppn	Accepted	Multiple passenger requests are handled before the shuttle returns.
pp	Rejected	The string ends while the shuttle is still in the moving/boarding stage.
ppp	Rejected	Multiple requests occur, but the shuttle cycle remains incomplete.
pnpn	Accepted	Two complete shuttle cycles are performed back-to-back.
pnp	Rejected	One cycle is completed, but the second cycle starts and does not finish.

All twelve cases behaved exactly as intended by the design: every Accepted case corresponds to a string that leaves the shuttle in the Waiting Area along at least one non-deterministic path, and every Rejected case corresponds to a string that ends mid-cycle or contains an invalid symbol.

9. Execution Screenshots / Outputs

A terminal screenshot could not be captured in this authoring environment; the console output reproduced below is the actual, unedited output of running `python3 shuttle_nfa.py`, and is provided in place of a screenshot. Students should replace this section with an actual screenshot of their own terminal run before submission.

OUTPUT:

```
=====
AIRPORT SHUTTLE NFA SIMULATOR
=====

Allowed input symbols:
p -> Passenger request
n -> No passenger request

Only lowercase letters are allowed!

Enter input string (or 'exit' to quit): pnpn

Input String: pnpn
Step 0: {'q0'}
Step 1: On 'p' -> {'q2', 'q1'}
Step 2: On 'n' -> {'q0', 'q3'}
Step 3: On 'p' -> {'q2', 'q1'}
Step 4: On 'n' -> {'q0', 'q3'}

RESULT: ACCEPTED

Do you want to try again? (y/n): y

Enter input string (or 'exit' to quit): pnpnp

Input String: pnpnp
Step 0: {'q0'}
Step 1: On 'p' -> {'q2', 'q1'}
Step 2: On 'n' -> {'q0', 'q3'}
Step 3: On 'p' -> {'q2', 'q1'}
Step 4: On 'n' -> {'q0', 'q3'}
Step 5: On 'p' -> {'q2', 'q1'}

RESULT: REJECTED

Do you want to try again? (y/n): n

Thank you!
```

10. Results and Validation

The simulator was run against all twelve test cases from Table 8.1. The table below summarises the outcome.

SUMMARY TABLE		
Input	Result	Description
eps	ACCEPTED	Empty string – shuttle idles in the Waiting Area and never leaves.
n	ACCEPTED	No request received while waiting – shuttle stays in the Waiting Area.
nnnn	ACCEPTED	Several idle cycles with no passenger request.
p	REJECTED	Single request – shuttle leaves for boarding, but the cycle is not yet closed.
pn	ACCEPTED	One request, followed by no further request – one full shuttle cycle.
ppn	ACCEPTED	A second request is served during boarding, then the cycle closes.
ppppn	ACCEPTED	Boarding is re-triggered by multiple passenger requests, then finally closes.
pp	REJECTED	Ends while still mid-cycle in the Terminal/Boarding stage.
ppp	REJECTED	Ends after additional passenger requests; boarding is still open.
pnpn	ACCEPTED	Two complete shuttle cycles performed back-to-back.
pnP	REJECTED	One complete cycle, then a second cycle starts but is not closed.
pN	REJECTED	Invalid symbol N – not in the lowercase alphabet {p, n}.

12 out of 12 test cases produced the intended result, giving full agreement between the hand-derived expected

behaviour of the automaton and the behaviour of its epsilon-closure-based simulation. This constitutes empirical validation that the transition function in Table 5.1 faithfully implements the operational description in the problem statement.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 NFA vs. an Equivalent DFA

An equivalent DFA could be produced via the subset construction, but it would need at least one extra composite state (e.g., {q1, q2}) to track the fact that, immediately after a p from q0, the automaton is simultaneously 'in transit' and 'able to board'. While a DFA is easier to implement in hardware-style logic (no backtracking or parallel path tracking needed), it obscures the natural structure of the problem: the NFA's states map one-to-one onto the shuttle's real physical states, whereas the DFA's composite states do not correspond to anything physically meaningful. For this reason the NFA is the more natural and more maintainable model for this problem, even though it requires a simulator capable of tracking a set of active states.

11.2 Trade-offs of the Epsilon-Transition Design

Modelling the automatic Terminal-to-Boarding and Returning-to-Waiting moves as ϵ -transitions keeps the automaton's alphabet clean (limited strictly to the two real signals p and n) at the cost of requiring an epsilon-closure step in the simulator. The alternative - folding these automatic moves into the p/n transitions directly - was tried first and rejected during design, because it caused the same input symbol to sometimes mean 'a new passenger request' and sometimes mean 'the automatic continuation of the previous move', which produced logically inconsistent acceptance behaviour once tested. The ϵ -transition design removes this ambiguity and was confirmed correct by the full test suite in Section 8.

12. Broader Considerations / SDG Relevance

This assignment is mapped to SDG 9 (Industry, Innovation and Infrastructure). Automated shuttle systems of the kind modelled here are a small but representative example of the formal verification techniques that underpin safe, efficient transport infrastructure. Modelling shuttle behaviour as a finite automaton before implementation allows unsafe or undefined sequences of operation (for example, a shuttle instructed to board passengers while mid-transit with no defined next state) to be identified and eliminated at the design stage, rather than discovered as a bug in a physical system. This kind of formal-methods thinking scales to more sophisticated transport-infrastructure controllers, including traffic signal logic, autonomous vehicle mode-switching, and airport ground-operations scheduling.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

A four-state NFA was successfully designed to recognise valid operational sequences of the airport shuttle system. The non-deterministic branch at the Passenger Boarding state was correctly modelled using a transition to a set of two states, and the automaton was validated through a Python simulation against twelve test cases with a 100% pass rate.

13.2 Limitations

- The model treats p and n as instantaneous, discrete signals and does not capture real-world timing (e.g., how long boarding takes).
- The automaton does not model failure conditions such as a stalled shuttle or a sensor fault.
- Acceptance is defined only by the final state after the string is consumed; it does not distinguish between multiple valid interpretations (paths) of the same accepted string.

13.3 Possible Improvements

- Extend the alphabet with additional signals (e.g., a fault signal F) and add corresponding error-handling states.

- Convert the NFA to an equivalent minimal DFA for comparison, and empirically measure simulation speed on both.
- Model maximum boarding capacity using a bounded counter, moving toward a pushdown

14. Individual Contribution of Group Members

[Fill in with each member's actual contribution before submission. A sample structure is provided below.]

Team Member	Registration No.	Contribution
D Gunavardhan	192372041	NFA design, transition table, report structure
Pusalapati Chandu	192372119	Python simulator, test-case execution and documentation
A Dileep Kumar	192372134	State diagram, GitHub upload, reflection note

15. References

1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.
2. Sipser, M. Introduction to the Theory of Computation (3rd ed.). Cengage Learning.
3. Graphviz - Graph Visualization Software. <https://graphviz.org/>
4. Python Software Foundation. Python 3 Language Reference. <https://docs.python.org/3/>
1. United Nations. Sustainable Development Goal 9: Industry, Innovation and Infrastructure. <https://sdgs.un.org/goals/goal9>

16. One-Page Individual Reflection

Team Member: Pusalapati Chandu

Registration No.: 192372119

Contribution: Python Simulator, Test-Case Execution and Documentation

Design and Development Decisions

My main contribution was implementing the NFA simulator using **Python 3 in Python IDLE**. I converted the NFA states and transition table into Python code. I implemented the `epsilon_closure()`, `nfa_step()`, and `simulate()` functions. The program accepts only lowercase p and n and displays the current states and final result.

Challenges Faced

The main challenge was handling multiple possible states in an NFA. I used Python sets to store the current states. Another challenge was implementing epsilon closure so that automatic transitions were followed without consuming an input symbol. I tested different input strings and invalid symbols to verify the program.

Learning Gained

I learned how to convert an NFA model into a working Python program. I improved my understanding of Python functions, sets, dictionaries, loops, input validation, and testing.

SDG Connection

This project is related to SDG 9 – Industry, Innovation and Infrastructure because the simulator models an airport transportation process and helps verify its possible operational sequences.

Contribution to Course Outcomes

The project helped me connect theoretical automata concepts with practical programming. I gained experience in implementing, testing, and documenting an NFA simulator using Python.